Use Case Overview

SENG300 | Online Multiplayer Game Platform — Project Iteration 1

This document provides a structured overview of all use cases defined for the Online Multiplayer Game Platform. Use cases are organized into five subsystems, each with its own diagram. Together they cover every player-facing interaction from first visit through account management, matchmaking, active gameplay, and profile settings.

About Use Case Diagrams

A use case diagram shows what a system does from the user's perspective, not how it is implemented. Each use case answers one question: what goal can an actor accomplish with the system?

What use cases define:

- Who interacts with the system (actors)
- What goals those actors can accomplish (use cases)
- The system boundary — what is inside vs. external
- Relationships between use cases (`<<include>>` for mandatory sub-flows, `<<extend>>` for conditional branches)

What use cases deliberately avoid:

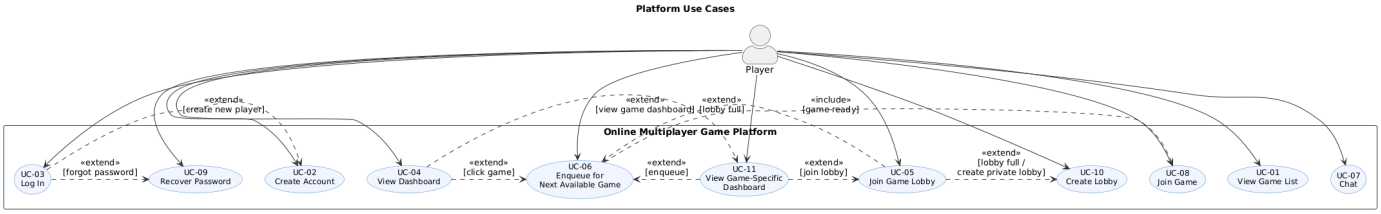
- Internal logic, algorithms, or data structures
- Execution order or control flow
- UI design details

Actor Summary

Actor	Description
Player	The primary actor across all subsystems. Represents any registered or unregistered user of the platform.
User Store (Stub)	External stub simulating the authentication/credential database.
Game Data (Stub)	External stub simulating persistent game statistics and leaderboard storage.

Subsystem 01 — Platform Use Cases

Diagram: `03.01 Platform UC Diagram.png`



This diagram defines the top-level use cases available on the platform. It represents the broadest view of what players can do and serves as the entry point that connects to all other subsystems.

Use Cases

ID	Name	Auth Required	Description
UC-01	View Game List	No	Player visits the welcome page and browses available games without authenticating. Read-only.
UC-02	Create Account	No	Player registers a new account. System validates input, checks username availability, creates the account, and optionally auto-logs in.
UC-03	Log In	No	Player submits credentials. Authentication stub returns SUCCESS or FAILURE. On success, player is routed to the dashboard.
UC-04	View Dashboard	Yes	Authenticated landing page showing available games, navigation to profile/leaderboard/settings, and current status.
UC-05	Join Game Lobby	Yes	Player selects a game and joins an existing lobby to coordinate and chat before a session begins.
UC-06	Enqueue for Next Available Game	Yes	Player enters the matchmaking queue for a game type and waits to be assigned to a session.
UC-07	Chat	Yes	Player sends and receives messages in a lobby or global context.
UC-08	Join Game	Yes	Player enters an active game session after being matched or invited.
UC-09	Recover Password	No	Player initiates password recovery from the login page. System simulates sending a recovery code.
UC-10	Create Lobby	Yes	Player creates a public or private lobby for a specific game type with configurable settings.

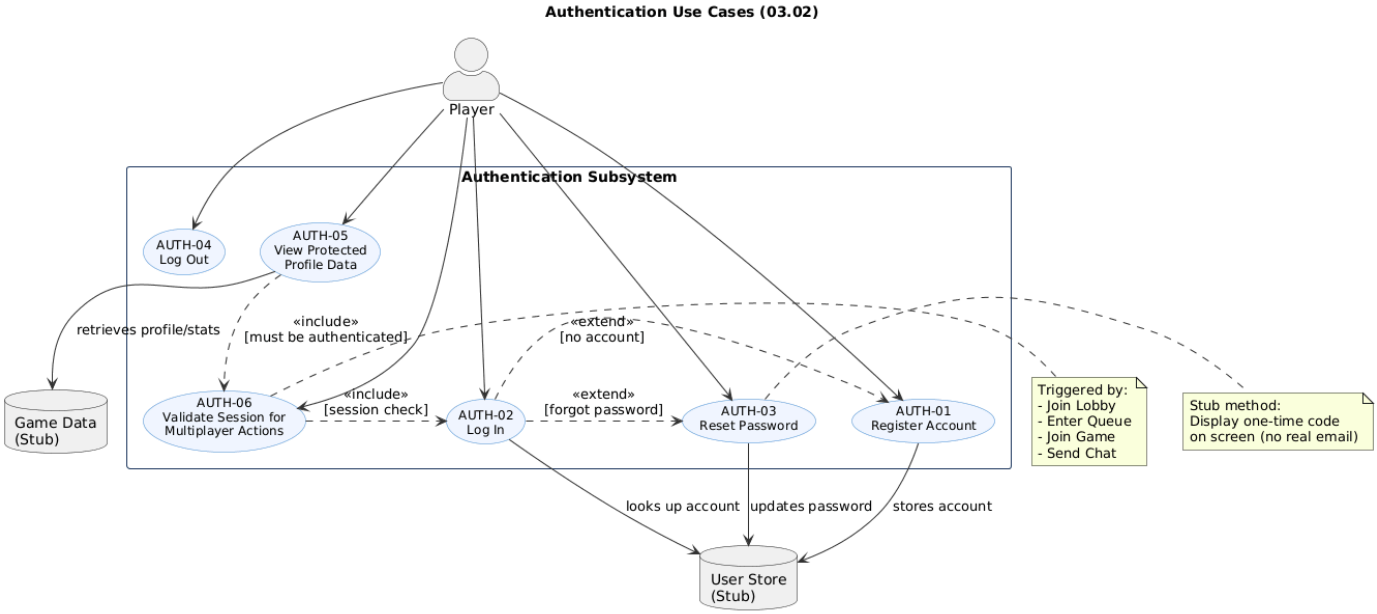
ID	Name	Auth Required	Description
UC-11	View Game-Specific Dashboard	Yes	Player views a dedicated dashboard for one game, including their stats, leaderboard, and matchmaking options.

Key Relationships

- **UC-03** `<<extend>>` **UC-09** — Player can trigger password recovery from the login failure path
- **UC-03** `<<extend>>` **UC-02** — Player can navigate to account creation from the login page
- **UC-04** `<<extend>>` **UC-06** — Clicking a game from the dashboard triggers the queue flow
- **UC-04** `<<extend>>` **UC-11** — Player can open a game-specific dashboard from the main dashboard
- **UC-05** `<<extend>>` **UC-10** — If a lobby is full, player is offered the option to create their own
- **UC-05** `<<extend>>` **UC-06** — If a lobby is full, player can enqueue instead
- **UC-11** `<<extend>>` **UC-06** — Game dashboard offers enqueue action
- **UC-11** `<<extend>>` **UC-05** — Game dashboard offers join lobby action
- **UC-06** `<<include>>` **UC-08** — When a match is found, joining the game session is a mandatory sub-flow

Subsystem 02 — Authentication Use Cases

Diagram: `03.02 Authentication UC Diagram.png`



This subsystem defines how players prove their identity, how sessions are managed, and how protected data is accessed. All multiplayer actions depend on an active authenticated session validated by AUTH-06.

Use Cases

ID	Name	Description
AUTH-01	Register Account	Creates a new account in the User Store Stub (HashMap or file). Validates format, checks for duplicate identifiers, and optionally auto-logs in.
AUTH-02	Log In	Looks up the account in the User Store Stub, verifies the password, creates an authenticated session, and navigates to the dashboard.
AUTH-03	Reset Password	Simulates password recovery by displaying a one-time code on screen (stub). Validates the code, enforces password rules, and updates the stored password.
AUTH-04	Log Out	Clears the authenticated session (sets `currentUser = null`) and returns to the welcome/login page.
AUTH-05	View Protected Profile Data	Verifies authentication and data visibility rules, then retrieves profile and stats from the Game Data Stub for display.
AUTH-06	Validate Session for Multiplayer Actions	Gate check performed before any multiplayer action (join lobby, enter queue, join game, send chat). Blocks expired or missing sessions.

Key Relationships

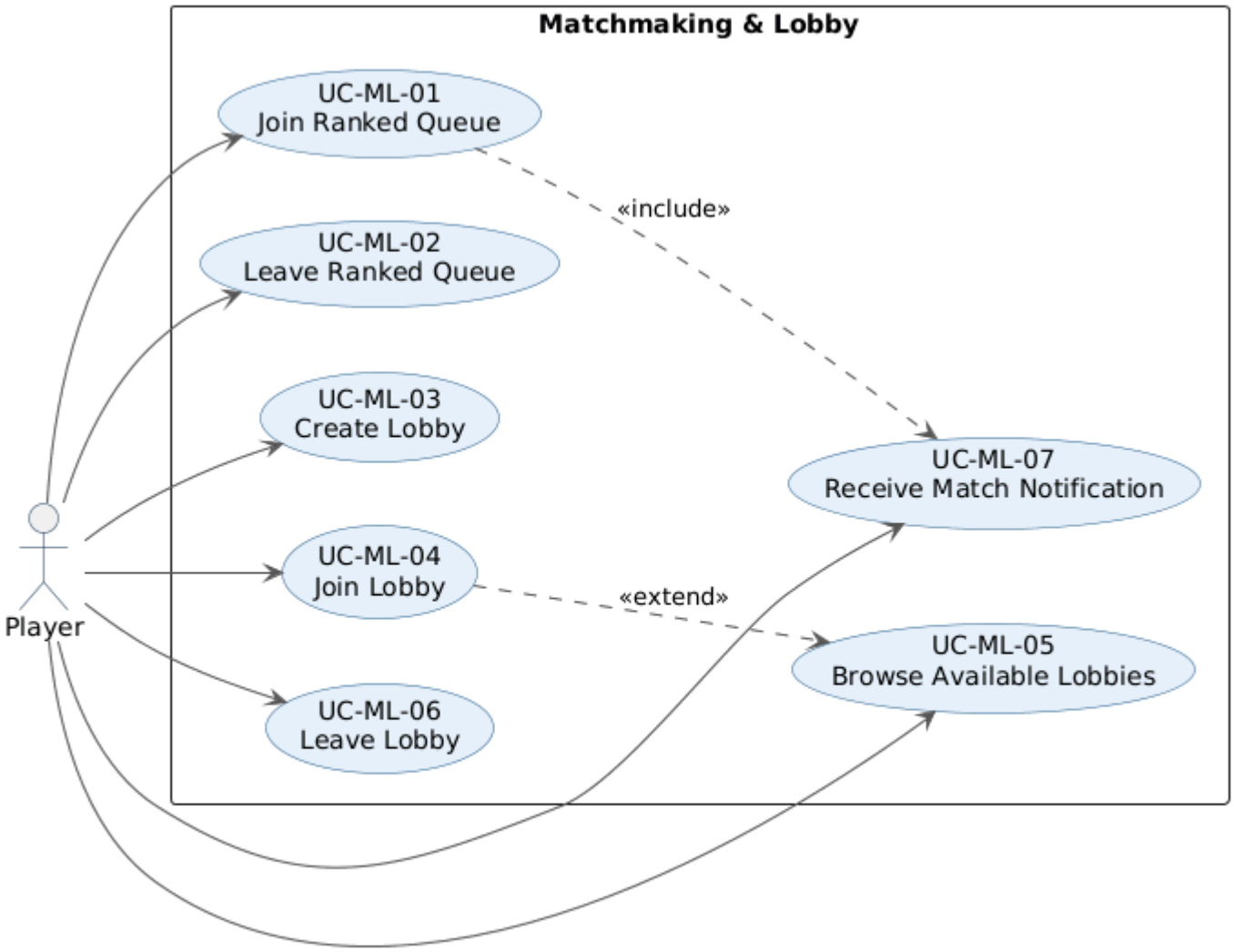
- **AUTH-02** `<<extend>>` **AUTH-03** — Player can trigger password reset from the login failure path
- **AUTH-02** `<<extend>>` **AUTH-01** — Player can navigate to registration from login
- **AUTH-05** `<<include>>` **AUTH-06** — Viewing protected data requires session validation
- **AUTH-06** `<<include>>` **AUTH-02** — Session check references the login state established by AUTH-02
- **AUTH-02**, **AUTH-03** → **User Store (Stub)** — Both look up and update accounts in the User Store
- **AUTH-05** → **Game Data (Stub)** — Profile and stats are retrieved from the Game Data Stub

Stub Notes

Stub	Purpose
User Store (Stub)	HashMap (in-memory) or file-backed store for account credentials
Game Data (Stub)	Provides profile and stats data for display without real persistence
Password recovery	Displays a one-time code on screen; no real email is sent
Session	Implemented as a simple in-memory variable (e.g., `currentUser`)

Subsystem 03 — Matchmaking & Lobby Use Cases

Diagram: `03.03 Matchmaking Lobby UC.png`



This subsystem defines how players get into a game — either through the ranked matchmaking queue or through manually created/joined lobbies. UC-ML-07 is the critical bridge between this subsystem and the Game Session subsystem.

Use Cases

ID	Name	Description
UC-ML-01	Join Ranked Queue	Player selects a game type and enters the ranked matchmaking queue. System checks eligibility (ban status, score constraints) against the Database stub before enqueuing.
UC-ML-02	Leave Ranked Queue	Player withdraws from the queue before a match is assigned. If a match has already been locked, the player cannot leave.

ID	Name	Description
UC-ML-03	Create Lobby	Player creates an unranked lobby for a specific game type. System adds the creator as the first member and displays the lobby view with chat.
UC-ML-04	Join Lobby	Player selects an existing lobby from the browse list and joins it, provided space is available.
UC-ML-05	Browse Available Lobbies	Player views a list of joinable lobbies for a given game type. Read-only; selecting one triggers UC-ML-04.
UC-ML-06	Leave Lobby	Player exits a lobby before the game session begins. Player is returned to the dashboard browsing state.
UC-ML-07	Receive Match Notification	System-triggered. When a match is found (ranked via `MatchMaking.matchPlayers()` or unranked via all-ready lobby), all matched players are notified and transitioned into the game session.

Key Relationships

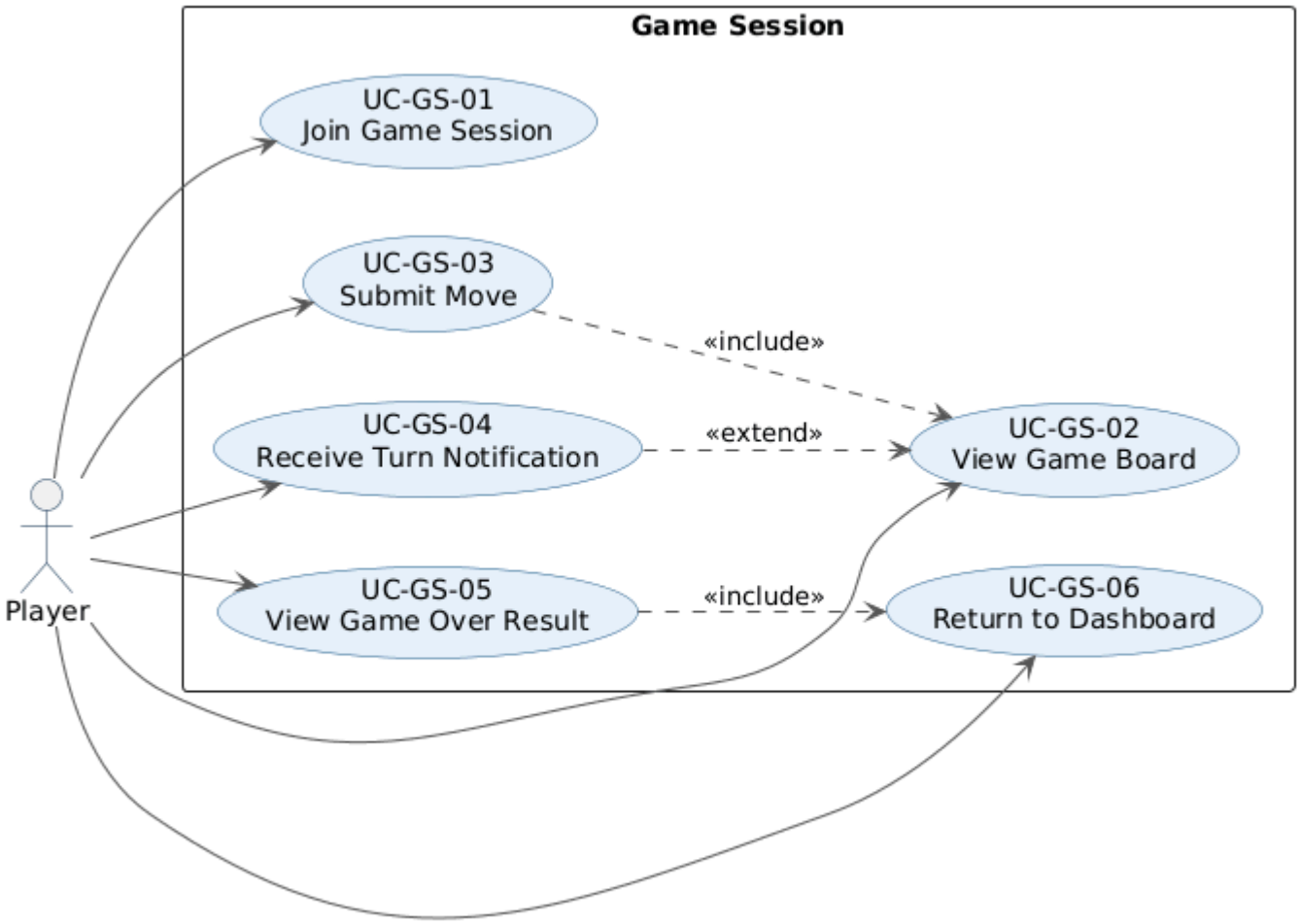
- **UC-ML-01** `<<include>>` **UC-ML-07** — Joining the queue eventually leads to receiving a match notification
- **UC-ML-04** `<<extend>>` **UC-ML-05** — Joining a lobby extends from browsing the lobby list
- **UC-ML-07** → **Game Session (UC-GS-01)** — Match notification triggers the game session join flow in the next subsystem

Alternate Flows

Situation	Response
Eligibility check fails (UC-ML-01)	Error message shown; player stays on dashboard
Queue already locked (UC-ML-02)	Player cannot leave; proceeds to game session
Lobby full (UC-ML-04)	Player notified; returned to browse view

Subsystem 04 — Game Session Use Cases

Diagram: `03.04 GameSession UC.png`



This subsystem covers everything that happens inside an active game: joining, viewing the board, submitting moves, receiving turn notifications, and seeing the result. It maps directly onto the platform's core turn engine and event-bus architecture.

Use Cases

ID	Name	Description
UC-GS-01	Join Game Session	Player enters an assigned game session. `GameSessionManager.startSession()` initialises the game and displays the initial board state via `PlayGame` and `GameGUI`.
UC-GS-02	View Game Board	Player sees the current board state. Triggered by `GameStateUpdate` events delivered through `EventBus` → `PlatformCore` → `GameGUI.renderBoard()`.
UC-GS-03	Submit Move	Player selects and confirms a move. `RuleValidator` validates it; if valid, `Game.handleAction()` applies it and publishes a `GameStateUpdate`. If invalid, an error is shown and the player retries.

ID	Name	Description
UC-GS-04	Receive Turn Notification	If the player is not on the game screen when their turn arrives, a banner or badge is pushed via `notifyPlayer()`. Clicking it navigates to the game screen and triggers UC-GS-02.
UC-GS-05	View Game Over Result	When `Game.isGameOver()` returns true, `StatsManager.recordResult()` updates stats and rank. `PlatformCore` fetches the updated data and displays win/loss/draw and rank to all session players.
UC-GS-06	Return to Dashboard	Player exits the result screen. `PlatformCore.returnToDashboard()` opens `MainGameDashboard` and sets player session state back to `Browsing`.

Key Relationships

- **UC-GS-03** `<<include>>` **UC-GS-02** — Submitting a move always results in a board update
- **UC-GS-04** `<<extend>>` **UC-GS-02** — Turn notification conditionally leads to a board view
- **UC-GS-05** `<<include>>` **UC-GS-06** — Viewing the result always includes the option to return to dashboard

Alternate Flows

Situation	Response
Invalid move (UC-GS-03)	Rejection reason displayed; player stays on their turn
Move ends the game (UC-GS-03)	Flow proceeds directly to UC-GS-05
Player not on game screen (UC-GS-04)	Notification banner pushed to current screen

Subsystem 05 — Profile, Stats & Settings Use Cases

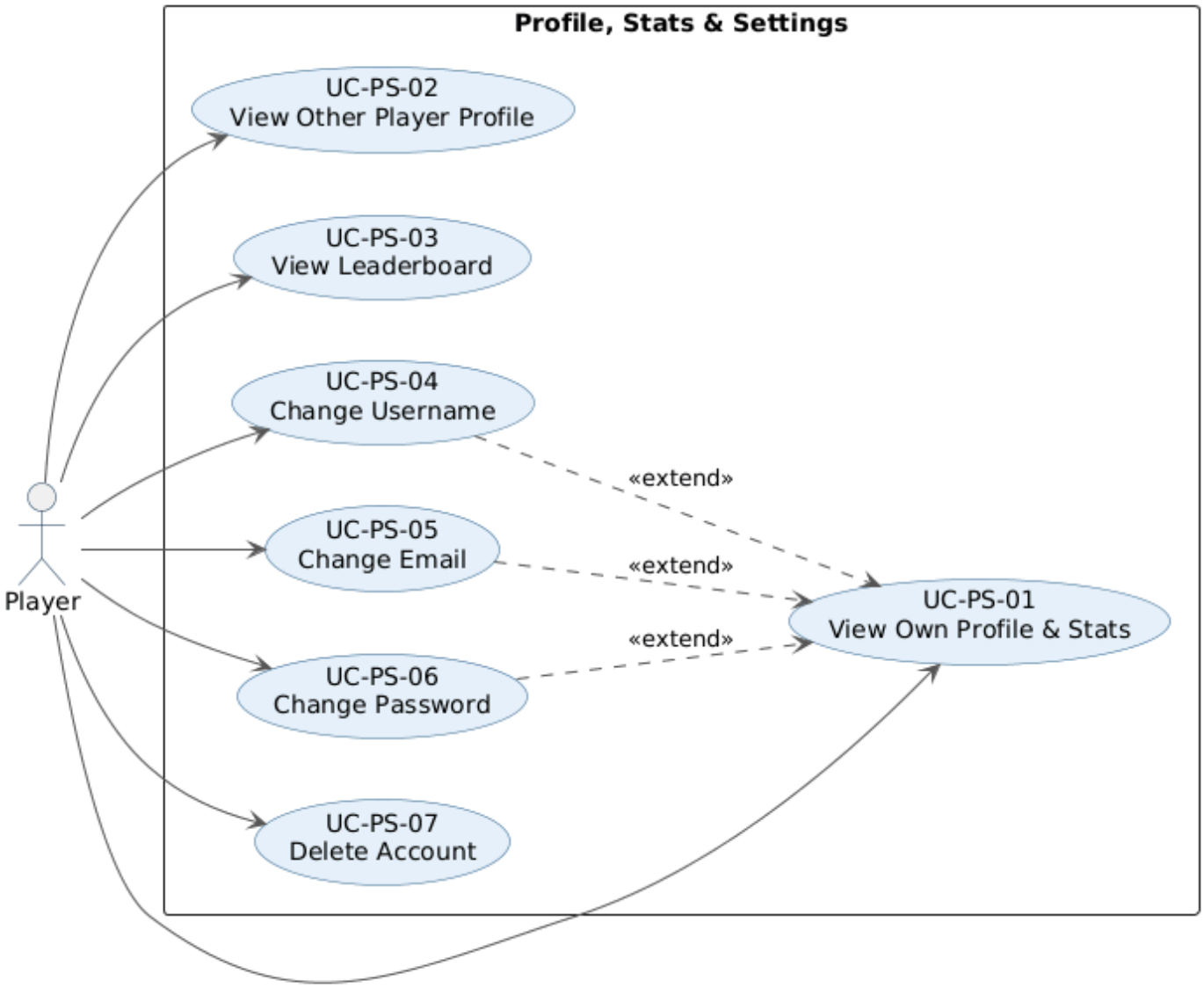


Diagram: `03.05 Profile Stats Settings UC.png`

This subsystem covers how players view and manage their own data and account. All use cases in this subsystem require authentication. The settings use cases (UC-PS-04 through UC-PS-07) all operate against the User Store Stub.

Use Cases

ID	Name	Description
UC-PS-01	View Own Profile & Stats	Player views their own stats per game type (wins, losses, ties, MMR, rank). Data fetched from the Database stub via `PlayerStats`.
UC-PS-02	View Other Player Profile	Player searches for or selects another player's username and views their public profile and per-game statistics.

ID	Name	Description
UC-PS-03	View Leaderboard	Player views top-ranked players for a selected game type via `Leaderboard.getTopPlayersByRank(gameType)`. Player can switch between game types.
UC-PS-04	Change Username	Player enters a new username. `Settings.changeUsername()` validates format, checks availability in `CredentialsDatabase`, and updates the User Store stub.
UC-PS-05	Change Email	Player enters a new email. `Settings.changeEmail()` validates email format and updates the User Store stub.
UC-PS-06	Change Password	Player provides their current password and a new one. `Settings.changePassword()` verifies against the bcrypt hash and updates the stored hash.
UC-PS-07	Delete Account	Player confirms account deletion with their password. `CredentialsDatabase.deleteExistingPlayer()` removes the account, the session is cleared, and the player is returned to the welcome page.

Key Relationships

- **UC-PS-04** `<<extend>>` **UC-PS-01** — Changing username can extend from the profile view
- **UC-PS-05** `<<extend>>` **UC-PS-01** — Changing email can extend from the profile view
- **UC-PS-06** `<<extend>>` **UC-PS-01** — Changing password can extend from the profile view

Alternate Flows

Situation	Response
Player not found (UC-PS-02)	"Player not found" shown; return to previous screen
Username taken (UC-PS-04)	`State.UsernameTaken` returned; player prompted to choose another
Invalid email format (UC-PS-05)	`State.EmailFormatWrong` returned; player re-enters
Current password wrong (UC-PS-06)	Error shown; player re-enters
Player in active game (UC-PS-07)	Deletion blocked; player must finish or leave the game first

> **Note:** UC-PS-06 covers the authenticated "change password" flow. For unauthenticated "forgot password," see AUTH-03.